Creating a Backup with Rsync

Rsync is a Linux based tool used for backup and file recovery. In this article, we will see how to set it up in Linux and also understand some basic operations.

A backup is the process of creating and storing copies of data that can be used to protect companies against data loss. An appropriate backup copy is stored in a distinct system or medium, such as a tape. This can be used to restore data in case of a disaster.

Rsync is a Linux based tool used for backup and file recovery. It transfers and synchronises files between a machine and an external hard drive, or across a network. Rsync makes the procedure more proficient by collating the modification dates and sizes of the files, and creates a backup only if needed.

Rsync is a set of rules that gives incredible adaptability for backing up and synchronising data. It is used locally as well as externally to back up files to various directories or can be configured to sync across the Internet to other hosts. Rsync is also able to take automatic backups. It can be used on Windows, but is only available through different plugins (such as Cygwin).

In this article, we will see how to set it up in Linux and also understand some basic operations. First, we need to install/update the Rsync client. On Red Hat distributions, the command is *yum install rsync* and on Debian it is *sudo apt-get install rsync*.

Uploading and downloading files using Rsync

We will start with the basics. The simplest usage of Rsync is:

```
rsync SRC DEST
```

Here, SRC is the source directory of files and DEST is the destination where the files are to be copied. While uploading any file to a remote server,

SRC should be on the local machine and DESC on the remote server.

We can also customise Rsync's behaviour with options. For example, we can use *-a*, which tells Rsync to preserve everything, and copy all the files in the directory recursively.

Uploading a folder to backup server

Give the following command to upload a folder to the backup server:

```
rsync -a local_directory username@
remote_server:/path/to/backup_
directory
```

If we want to delete some extra files from a backup server, the *--delete* option tells Rsync to delete files in the backup directory that are not also present in the SRC folder:

```
rsync -a --delete local_directory
username@remote_server:/path/to/
backup_directory
```

We have successfully created a backup of our files.

Next, if we want to download the backed up files to our local machine, we can simply reverse the SRC and DEST locations, and the files will be downloaded from the remote server:

```
rsync -a username@remote_server:/path/
to/backup_directory local_directory
```

Preview before changes are made

Rsync has a feature to help you preview the changes that will happen before finalising them in your directories. *--dry-run* or the *-n* options will tell Rsync to not execute any file transfers, but show details of the file transfers that will happen instead:

```
rsync -av --dry-run local_directory
username@remote_server:/path/to/backup_
directory
```

The *-v* option can be used if we want to see a verbose output. We can also specify a log file to store the result:

```
rsync -av --dry-run local_directory
username@remote_server:/path/to/backup_
directory --log-file=/path/to/log_file
```

Using the Rsync protocol

In all of the above examples, Rsync uses a remote shell like SSH as the transport method. On the other hand, you can connect to a remote Rsync daemon directly by specifying the Rsync protocol for the remote path:

```
rsync -a rsync://username@remote_
server:/module_name/file_to_download
local_directory
```

If you are looking for a tool to help you make complex file transfers, Rsync may be what you are looking for. This article only covers the basics; you can find many more Rsync commands on the developer's documentation site. END

By: Neetesh Mehrotra

The author works at NIIT Technologies as a senior test engineer. His areas of interest are Java development and automation testing.